

Ansible for the ZMS Lab

A Practical Training Course
& Complete Codebase Reference

*Configuration management for a dynamically
discovered AWS estate*

Linux (Ubuntu & Amazon Linux) and Windows Server

Repository	zms-ansible-code
Control model	Push — SSH (Linux), WinRM (Windows)
Discovery	EC2 dynamic inventory (amazon.aws.aws_ec2)
Secrets	AWS Secrets Manager (none stored in Git)
Audience	Operators new to Ansible; estate maintainers

How to read this guide. Part I teaches Ansible from first principles — no prior experience assumed. Part II explains the architecture of this repository. Part III documents *every file*, directory by directory. Part IV is the day-to-day operations manual. Part V is the hands-on training course: modules, labs, and a self-check quiz with answers. The appendices are quick references you will return to.

Accuracy note. Every configuration value, module name, default, and secret identifier in this guide was transcribed directly from the repository source. Where the text quotes a file, it quotes the file as it actually exists.

Contents

Part I — Ansible Fundamentals	4
1.1 What Ansible is, and the problem it solves	4
The push model used in this repository	4
1.2 The core vocabulary	4
1.3 How a playbook actually runs	5
1.4 Idempotency and the changed state	6
1.5 Installing Ansible and collections	6
1.6 The core objects, illustrated	7
Module — the unit of work	7
Task — a module call with metadata	7
Inventory — who to manage	7
Variables — named values (and precedence)	7
Facts — data Ansible discovers about a host	8
Conditionals — when	8
Loops — loop	8
Register — capture a task's result	9
Handlers — act only when something changed	9
Blocks — group tasks and handle errors	9
Templating and filters (Jinja2)	10
Lookups — fetch data at run time (on the control node)	10
Play — bind a group of hosts to tasks and roles	10
Playbook — one or more plays in a file	10
Role — a reusable, structured bundle	11
Collection — a versioned bundle of modules/plugins/roles	11
Tags — run a slice of a playbook	12
How the objects relate in this repository	12
Part II — Repository Architecture	13
2.1 The big picture	13
2.2 Directory and file map	13
2.3 ansible.cfg — the engine configuration	14
2.4 requirements.yml — the collections	15
2.5 Dynamic inventory — inventory/aws_ec2.yml	16
2.6 Variables — the group_vars/ directory	17
group_vars/all.yml — applies to every host	17
group_vars/os_linux.yml — SSH connection + login	18
group_vars/os_windows.yml — WinRM connection (consolidated secret)	18
host_vars/ — per-host overrides	19
2.7 Variable precedence (why the right value wins)	19
2.8 The baseline role	19
roles/baseline/tasks/main.yml — the dispatch entry point	19
roles/baseline/tasks/linux.yml	19
roles/baseline/tasks/windows.yml	20
roles/baseline/defaults/main.yml	20
roles/baseline/meta/main.yml and README.md	20
2.9 The control-node lifecycle: bootstrap.yml and reconverge.sh	21
scripts/reconverge.sh	21

bootstrap.yml	21
2.10 The estate playbook: site.yml	22
Part III — The Task Playbooks	25
3.1 windows-adds.yml — build a new AD forest	25
3.2 windows-domain-join.yml — join hosts to the domain	25
3.3 windows-iis.yml — enable the IIS web server	26
3.4 windows-share.yml — create a read-only SMB share	26
3.5 windows-python.yml — install Python via Chocolatey	26
3.6 windows-zms-enforcer.yml — install the Zscaler ZMS Enforcer	26
3.7 ubuntu-setup.yml — update + common packages (Ubuntu)	28
3.8 ubuntu-apache2.yml — Apache2 service (Ubuntu)	28
3.9 ubuntu-mysql.yml — MySQL service (Ubuntu)	28
3.10 amazonlinux-setup.yml — update + common packages (Amazon Linux)	28
3.11 amazonlinux-httpd.yml — Apache (httpd) service (Amazon Linux)	29
3.12 amazonlinux-mysql.yml — MySQL service (Amazon Linux)	29
Part IV — Operating the Estate	30
4.1 The everyday commands	30
4.2 Targeting model recap	30
4.3 Secrets the repository expects	31
4.4 Drift cadence	31
4.5 Linting and CI	31
4.6 What .gitignore protects	32
Part V — Hands-On Training Course	33
Module 1 — Orientation and a safe first run	33
Module 2 — Reading the inventory and groups	33
Module 3 — Connectivity (ping) to both OS families	33
Module 4 — A tagged, scoped converge	33
Module 5 — Install a service on selected hosts	34
Module 6 — A consolidated update with tags	34
Module 7 — A secret-backed Windows workflow	34
Module 8 — Capstone	34
5.1 Self-check quiz	35
5.2 Quiz answers	35
Appendix A — Command Cheat Sheet	36
Appendix B — --limit pattern arithmetic	37
Appendix C — Glossary	38
Appendix D — Troubleshooting	39
Appendix E — File Index	40

Part I — Ansible Fundamentals

1.1 What Ansible is, and the problem it solves

Ansible is an open-source **configuration management and automation** engine. You describe the *desired state* of a fleet of machines in human-readable YAML, and Ansible makes each machine match that description. You do not write step-by-step scripts that say “run this command, then that command”; instead you declare “this package should be present, this service should be running, this file should contain this text,” and Ansible figures out what (if anything) needs to change.

Three properties make this approach powerful:

Agentless. There is no permanent daemon to install on the machines you manage. Ansible connects on demand using protocols the machines already speak — **SSH** for Linux and **WinRM** (or SSH) for Windows — runs its work, and disconnects. The only requirements on a managed Linux host are an SSH server and Python; on Windows, a configured WinRM listener and PowerShell.

Idempotent. Running the same playbook twice is safe. A correctly written task checks the current state first and only acts if reality differs from the goal. The first run might install a package; the second run sees it already installed and reports ok instead of changed. This is what lets you run Ansible repeatedly as a *drift-correction* tool, not just a one-time installer.

Declarative and readable. The YAML files double as documentation. A new team member can read a playbook and understand what the estate looks like without running anything.

The push model used in this repository

Ansible can run in two broad modes. In **pull** mode each host periodically fetches and applies its own configuration. In **push** mode a central **control node** connects out to the managed hosts and applies configuration to them. This repository is built entirely around the **push** model: a single control node (an EC2 instance inside the VPC) holds this Git repository and pushes configuration to every other instance it discovers.

1.2 The core vocabulary

You will meet these terms throughout the guide. Read this section once; refer back as needed.

Control node. The machine that runs `ansible` / `ansible-playbook`. It holds the repository, the inventory, and the credentials. In this lab it is an EC2 instance inside the VPC with IAM rights to describe EC2 instances and read secrets.

Managed node (host). Any machine Ansible configures. Here: the Ubuntu, Amazon Linux, and Windows Server EC2 instances discovered dynamically.

Inventory. The list of managed nodes and how they are grouped. It can be a static file or, as in this repo, generated dynamically by querying a cloud provider. Hosts are organized into **groups**.

Module. A small, single-purpose unit of work that Ansible ships to a host and executes — for example `ansible.builtin apt` (manage Debian packages), `ansible.windows.win_service` (manage a Windows service), or `amazon.aws.aws_ec2` (the inventory plugin). Modules are idempotent: you state the desired end state and the module reconciles it.

Collection. A versioned bundle of modules, plugins, and roles distributed through Ansible

Galaxy — for example `amazon.aws`, `ansible.windows`, `community.mysql`. Modern Ansible refers to modules by their **fully-qualified collection name (FQCN)**, e.g. `ansible.builtin.copy` or `chocolatey.chocolatey.win_chocolatey`.

Task. A single call to a module with its parameters, plus optional metadata (a name, a when condition, tags, loop, etc.).

Play. A mapping of a *group of hosts* to an ordered *list of tasks* (and/or roles). A play names its hosts: and the tasks that should run on them.

Playbook. A YAML file containing one or more plays. Running a playbook executes its plays top to bottom.

Role. A reusable, structured bundle of tasks, variables, files, templates, and defaults, laid out in a conventional directory structure so it can be dropped into any play. This repository ships one role, `baseline`.

Variable. A named value. Variables can be defined in many places — `group_vars`, `host_vars`, inside a play, on the command line with `-e`, in role defaults — and Ansible merges them following a fixed **precedence** order.

Facts. Variables Ansible gathers automatically about a host at the start of a play (when `gather_facts: true`): its OS family, distribution, network interfaces, and so on. `ansible_os_family` (“Debian”, “RedHat”, “Windows”) is the fact this repository leans on most.

Handler. A task that runs only when *notified* by another task that reported a change — typically used to restart a service after its config file changes. (This repo keeps service management inline rather than using handlers, for simplicity.)

Templating (Jinja2). Ansible evaluates `{{ ... }}` expressions using the Jinja2 templating language. This is how variables, lookups, and filters are embedded in YAML, e.g. `"{{ rolling_batch | default('100%') }}"`.

Lookup. A plugin that fetches data at runtime *on the control node*, written as `lookup('name', ...)`. This repo uses `lookup('amazon.aws.aws_secret', ...)` to read credentials from AWS Secrets Manager and `lookup('ansible.builtin.env', ...)` to read an environment variable.

Connection plugin. The transport Ansible uses to reach a host: `ssh` for Linux, `winrm` for Windows, or `local` for the control node acting on itself.

Privilege escalation (become). Running tasks as a more privileged user — `sudo` to root on Linux. Controlled by `become`, `become_method`, and `become_user`.

Tag. A label attached to tasks or roles so you can run (or skip) a subset with `--tags / --skip-tags`.

Check mode (--check) and diff (--diff). A dry run that reports what *would* change without changing anything; `--diff` additionally shows the textual difference. This is the safe way to preview a converge.

1.3 How a playbook actually runs

Understanding the execution model prevents most beginner confusion.

1. **Parse and load.** Ansible reads `ansible.cfg`, loads the inventory (running the dynamic inventory plugin if configured), and reads variable files such as `group_vars/`.
2. **Per play:** Ansible selects the hosts matching the play’s `hosts:` pattern.
3. **Gather facts** (if enabled) by connecting to each host and collecting its system information into `ansible_*` variables.
4. **Execute tasks in order.** For each task, Ansible runs it across all selected hosts *before* moving to the next task. The number of hosts worked on simultaneously is the **fork** count (forks in `ansible.cfg`).

5. **Report a status per host per task:** ok (already correct), changed (action taken), skipping (a when condition was false), failed, or unreachable. A failing host is dropped from the remainder of the play by default.
6. **Summarize** with a recap line per host counting ok/changed/failed/etc.

The `serial:` keyword changes step 4's batching across hosts — instead of all hosts at once, Ansible processes them in batches (used in this repo for rolling, blast-radius-limited updates). `max_fail_percentage` aborts the play if too many hosts in a batch fail.

1.4 Idempotency and the changed state

Because Ansible reports changed only when it actually modified something, a *steady-state* converge of a healthy estate should report all ok and zero changed. A non-zero changed count on an unchanged system usually signals a non-idempotent task (for example, a raw command that always runs). This is why this repository marks read-only commands with `changed_when: false` — so that checking the SQLite/Python version or running `needs - restarting` never falsely reports a change.

1.5 Installing Ansible and collections

On the control node, Ansible core provides the engine and the `ansible.builtin` modules. Everything else — AWS, Windows, Chocolatey, MySQL modules — arrives as **collections** listed in `requirements.yml` and installed with:

```
ansible-galaxy collection install -r requirements.yml
```

Collections install into the path named by `collections_path` in `ansible.cfg` (here, the `repo-local/collections/` directory, which is git-ignored). The control node's own bootstrap playbook performs this install automatically; see Chapter 2.9.

1.6 The core objects, illustrated

This section catalogues each Ansible object with a small, self-contained example. The examples are deliberately generic — they use illustrative host names like `web01` and a static inventory so each idea stands alone. This repository uses a *dynamic* inventory and FQCN module names throughout; the cross-references point to where each object appears in the real code.

Module — the unit of work

A module performs one idempotent action on a host. You can call one directly (“ad-hoc”) without writing a playbook:

```
# Ensure nginx is installed on every host in the 'web' group, escalating to root
ansible web -m ansible.builtin apt -a "name=nginx state=present" --become
```

`-m` names the module, `-a` passes its arguments. The same module, expressed as a task inside a playbook, is shown next. In this repo, modules such as `ansible.builtin.apt`, `ansible.windows.win_service`, and `chocolatey.chocolatey.win_chocolatey` do all the real work.

Task — a module call with metadata

A task wraps a module call with a human-readable name and optional controls (when, tags, loop, become, register, notify):

```
- name: Install nginx
  ansible.builtin.apt:
    name: nginx
    state: present
    become: true
    tags: [web]
```

`name:` is what prints in the run output; the module key (`ansible.builtin.apt`) and its parameters describe the desired state.

Inventory — who to manage

The inventory lists hosts and groups. A static INI inventory:

```
[web]
web01.example.com
web02.example.com

[db]
db01.example.com ansible_host=10.0.0.5

# a group of groups
[prod:children]
web
db
```

Here `web` and `db` are groups, `prod` contains both, and `db01` overrides its connection address. This repo replaces the static file with a **dynamic** inventory that builds these groups from EC2 tags (see Chapter 2.5).

Variables — named values (and precedence)

Variables parameterise tasks. Defined once, referenced with `{{ }}`:

```
# group_vars/web.yml - applies to every host in the 'web' group
http_port: 8080
```

```
- name: Render the listen port
  ansible.builtin.debug:
    msg: "nginx will listen on {{ http_port }}"
```

The same variable can be set in several places; the most specific/explicit wins. From lowest to highest priority: role defaults/ → inventory group_vars → play vars: → -e on the command line. So `ansible-playbook site.yml -e http_port=9090` overrides everything (see Chapter 2.7).

Facts — data Ansible discovers about a host

When `gather_facts: true`, Ansible collects `ansible_*` variables before running tasks:

```
- name: Show what we are running on
  ansible.builtin.debug:
    msg: "{{ ansible_distribution }} {{ ansible_distribution_version }}
         (family: {{ ansible_os_family }})"
```

`ansible_os_family` — “Debian”, “RedHat”, or “Windows” — is the fact this repo branches on most often.

Conditionals — when

A task runs only if its `when: expression` is true:

```
- name: Install http (Debian only)
  ansible.builtin.apt:
    name: http
    state: present
  when: ansible_os_family == "Debian"
```

The Linux/Windows split in `site.yml` and the “skip non-Debian/non-RedHat” guards in the task playbooks are exactly this pattern.

Loops — loop

Repeat a task over a list, with `{{ item }}` as the current element:

```
- name: Install several packages
  ansible.builtin.package:
    name: "{{ item }}"
    state: present
  loop:
    - git
    - curl
    - vim
```

(For package modules you can also pass the whole list to `name:` directly, which this repo does in the `*-setup` playbooks — fewer transactions.)

Register — capture a task's result

register stores a task's output in a variable for later use:

```
- name: Is nginx active?
  ansible.builtin.command: systemctl is-active nginx
  register: nginx_state
  changed_when: false           # a read-only check is never a "change"

- name: Report the state
  ansible.builtin.debug:
    msg: "nginx is {{ nginx_state.stdout }}"
```

changed_when: false is important for read-only commands — this repo uses it for needs-restarting, python --version, and the inventory-graph check.

Handlers — act only when something changed

A handler is a task that runs **only if notified** by a task that reported changed, and only once, at the end of the play:

```
tasks:
- name: Deploy the nginx config
  ansible.builtin.template:
    src: nginx.conf.j2
    dest: /etc/nginx/nginx.conf
    notify: Restart nginx

handlers:
- name: Restart nginx
  ansible.builtin.service:
    name: nginx
    state: restarted
```

If the template is unchanged, the restart does not happen — avoiding needless service bounces. (This repo manages services inline for simplicity rather than via handlers.)

Blocks — group tasks and handle errors

A block groups tasks so shared directives (when, become, tags) apply to all of them, with optional rescue (on error) and always (regardless):

```
- name: Configure the internal repo (Debian)
  when: ansible_os_family == "Debian"
  block:
    - name: Install the signing key
      ansible.builtin.get_url:
        url: "{{ repo_base_url }}/keys/internal.gpg"
        dest: /usr/share/keyrings/internal.gpg
    - name: Add the repository
      ansible.builtin.apt_repository:
        repo: "deb [signed-by=/usr/share/keyrings/internal.gpg] {{ repo_base_url }}/deb
        ↪ stable main"
        state: present
  rescue:
    - name: Note the failure
      ansible.builtin.debug:
```

```
msg: "Internal repo configuration failed on {{ inventory_hostname }}"
```

site.yml uses exactly this block + when shape for the Debian repo setup.

Templating and filters (Jinja2)

{{ }} expressions are evaluated by Jinja2. **Filters** (after a |) transform values:

```
# 'default' supplies a fallback; the second arg 'true' also catches empty strings
region:    "{{ lookup('ansible.builtin.env', 'AWS_REGION') | default('eu-west-3', true)
↪ }}"
# 'from_json' parses a JSON string into an object you can index
creds:     "{{ secret_json | from_json }}"
username:  "{{ creds.username }}"
# other common filters: | length, | join(','), | bool
```

Lookups — fetch data at run time (on the control node)

A lookup pulls external data during templating. This repo's security model rests on it:

```
# Read a secret from AWS Secrets Manager at run time — nothing stored in Git
db_password: "{{ lookup('amazon.aws.aws_secret', 'app/db-password', region='eu-west-3')
↪ }}"
```

Lookups run on the **control node**, not the managed host (see the WinRM and DSRM credential examples in Chapters 2.6 and 3.1).

Play — bind a group of hosts to tasks and roles

A play maps hosts: to an ordered set of roles and tasks, plus play-wide settings (become, gather_facts, serial):

```
- name: Configure the web tier
  hosts: web
  become: true
  gather_facts: true
  roles:
    - common
  tasks:
    - name: Install nginx
      ansible.builtin.apt:
        name: nginx
        state: present
```

Playbook — one or more plays in a file

A playbook is a YAML file of plays, run top to bottom:

```
---
- name: Web tier
  hosts: web
  roles: [nginx]

- name: Database tier
  hosts: db
```

```
roles: [mysql]
```

```
ansible-playbook site.yml --check --diff  # preview
ansible-playbook site.yml                 # apply
```

site.yml in this repo is precisely this: one playbook, two plays (Linux and Windows).

Role — a reusable, structured bundle

A role packages tasks, variables, templates, and handlers in a conventional directory layout so it can be dropped into any play:

```
roles/nginx/
├── tasks/main.yml           # entry point – what the role does
├── handlers/main.yml       # handlers the tasks can notify
├── templates/nginx.conf.j2 # Jinja2 templates
├── files/                  # static files to copy
├── defaults/main.yml       # default variables (lowest precedence)
├── vars/main.yml           # role variables (higher precedence)
└── meta/main.yml           # metadata + role dependencies
```

```
# roles/nginx/tasks/main.yml
- name: Install nginx
  ansible.builtin.apt:
    name: nginx
    state: present
  notify: Restart nginx
```

```
# use it in a play – Ansible runs roles/nginx/tasks/main.yml automatically
- hosts: web
  roles:
    - nginx
```

This repo's baseline role (Chapter 2.8) follows the same layout, using tasks/main.yml to dispatch to linux.yml or windows.yml.

Collection — a versioned bundle of modules/plugins/roles

Collections distribute modules through Ansible Galaxy. You declare them in requirements.yml and reference their modules by **fully-qualified collection name** (FQCN):

```
# requirements.yml
collections:
  - name: community.mysql
    version: ">=3.0.0,<4.0.0"
```

```
# a module from that collection, referenced by FQCN
- name: Create a database
  community.mysql.mysql_db:
    name: appdb
    state: present
```

```
ansible-galaxy collection install -r requirements.yml
```

The eight collections this repo depends on are listed in Chapter 2.4.

Tags — run a slice of a playbook

Tags label tasks (or roles) so you can run or skip subsets:

```
- name: Install baseline packages
  ansible.builtin.package:
    name: htop
    state: present
    tags: [packages]
```

```
ansible-playbook site.yml --tags packages      # only tagged tasks
ansible-playbook site.yml --skip-tags packages # everything except them
```

How the objects relate in this repository

Object	Where it lives in zms-ansible-code
Inventory	inventory/aws_ec2.yml (dynamic, from EC2 tags)
Group variables	group_vars/all.yml, os_linux.yml, os_windows.yml
Collections	requirements.yml
Role	roles/baseline/
Playbook (multi-play)	site.yml
Playbook (single task)	the twelve files in playbooks/
Module (FQC�N)	every ansible.*/community.*/amazon.* task
Lookup	aws_secret / env lookups in vars and playbooks
Tags	baseline, packages, update

In one sentence: the **inventory** decides *who*, **group_vars** decide *how to connect*, **collections** provide the *tools*, **roles** and **playbooks** (made of **plays**, made of **tasks**, made of **modules**) decide *what to do*, and **lookups** fetch the *secrets* needed along the way.

Part II — Repository Architecture

2.1 The big picture

This repository is the **control repository** — the single source of truth the control node pulls and applies. The control node sits inside the AWS VPC, holds an IAM role that can *describe* EC2 instances and *read* specific secrets, and reaches managed hosts over their private IP addresses. Configuration flows in one direction: from this repo, through the control node, out to the estate.

The estate is **discovered, not enumerated**. Nothing in this repo lists individual server names. Instead, the dynamic inventory queries EC2 for running, Terraform-managed instances and sorts them into groups derived from their tags. The OS tag produces `os_linux` / `os_windows`; the Role tag produces `role_web`, `role_dc`, and so on; the Environment tag produces `env_prod`, `env_dev`, etc. Playbooks then target these groups rather than hostnames.

Three ideas recur throughout and are worth holding in mind:

1. **Tag-driven targeting.** Groups come from cloud tags, so adding a correctly tagged instance automatically places it in the right groups.
2. **OS-family branching.** Tasks select Debian (`apt`), RedHat (`dnf`), or Windows behaviour from the `ansible_os_family` fact, so one repo serves a mixed fleet.
3. **Secrets at runtime.** No password or key is stored in Git. Credentials are fetched from AWS Secrets Manager during the run via the `aws_secret` lookup.

2.2 Directory and file map

```
zms-ansible-code/
├── ansible.cfg                # engine configuration (the control knobs)
├── requirements.yml          # the Galaxy collections this repo needs
├── inventory/
│   └── aws_ec2.yml           # EC2 dynamic-inventory plugin configuration
├── group_vars/
│   ├── all.yml               # variables applied to every host
│   ├── os_linux.yml          # SSH connection + login settings (Linux)
│   └── os_windows.yml        # WinRM connection settings + Secrets lookup
├── host_vars/
│   └── .gitkeep              # placeholder; rare per-host overrides live here
├── roles/
│   └── baseline/             # one cross-platform role (timezone, packages)
│       ├── tasks/
│       │   ├── main.yml      # OS dispatch entry point
│       │   ├── linux.yml     # Linux baseline tasks
│       │   └── windows.yml    # Windows baseline tasks
│       ├── defaults/main.yml # role default variables
│       ├── meta/main.yml     # role metadata (Galaxy info)
│       └── README.md          # role documentation
├── scripts/
│   └── reconverge.sh          # cloud-init / systemd-timer entry point
├── playbooks/                # the task playbooks (Part III)
│   ├── windows-adds.yml
│   └── windows-domain-join.yml
```

```

├── windows-iis.yml
├── windows-share.yml
├── windows-python.yml
├── windows-zms-enforcer.yml
├── ubuntu-setup.yml
├── ubuntu-apache2.yml
├── ubuntu-mysql.yml
├── amazonlinux-setup.yml
├── amazonlinux-httpd.yml
├── amazonlinux-mysql.yml
├── bootstrap.yml           # the control node configuring itself (localhost)
├── site.yml               # the estate-wide push playbook
├── README.md              # repository quick reference
├── .ansible-lint           # lint profile
├── .yamllint              # YAML style rules
├── .pre-commit-config.yml  # local git hooks (lint on commit)
├── .github/workflows/lint.yml # CI: lint on push / PR
├── .gitignore             # never-commit list (collections, keys, caches)

```

The remainder of Part II walks the foundational files: configuration, inventory, variables, the role, the control-node lifecycle, and the estate playbook. Part III covers the twelve task playbooks individually.

2.3 ansible.cfg — the engine configuration

`ansible.cfg` is read automatically when `ansible/ansible-playbook` runs from the repository root. Every setting here is deliberate. The file as it exists:

```

[defaults]
inventory          = inventory/aws_ec2.yml
roles_path         = roles
collections_path   = collections
host_key_checking  = False
retry_files_enabled = False
forks              = 20
interpreter_python  = auto_silent
stdout_callback    = yaml
callbacks_enabled  = profile_tasks
fact_caching       = jsonfile
fact_caching_connection = /var/tmp/ansible_facts
fact_caching_timeout = 3600
vault_password_file = .vault_pass

[inventory]
enable_plugins = aws_ec2

[connection]
pipelining = True

[ssh_connection]
ssh_args = -o ControlMaster=auto -o ControlPersist=60s
control_path = /tmp/ansible-%%r@%%h:%%p

[privilege_escalation]
become = True
become_method = sudo

```

```
become_user = root
```

Setting by setting:

- **inventory** = **inventory/aws_ec2.yml** — use the EC2 dynamic inventory by default, so plain `ansible-playbook site.yml` already targets the live estate.
- **roles_path** = **roles** — find roles in the `repo-local/roles/` directory.
- **collections_path** = **collections** — install/look for collections in the `repo-local/collections/` directory (git-ignored).
- **host_key_checking** = **False** — do not prompt to verify SSH host keys. Cloud instances are ephemeral and churn host keys; prompting would block unattended runs. (A deliberate trade-off appropriate to a lab estate.)
- **retry_files_enabled** = **False** — do not litter the repo with `.retry` files after partial failures.
- **forks** = **20** — work on up to 20 hosts in parallel; ample for a few-dozen-host estate.
- **interpreter_python** = **auto_silent** — auto-detect the remote Python interpreter without printing a warning.
- **stdout_callback** = **yaml** — render task output as readable YAML.
- **callbacks_enabled** = **profile_tasks** — print per-task timing, useful for spotting slow tasks.
- **fact_caching** = **jsonfile / fact_caching_connection / fact_caching_timeout** — cache gathered facts as JSON under `/var/tmp/ansible_facts` for one hour, so repeated runs can skip re-gathering.
- **vault_password_file** = **.vault_pass** — where to read the Ansible Vault password (the file itself is git-ignored and sourced from the secret store at run time).
- **[inventory] enable_plugins** = **aws_ec2** — enable only the AWS EC2 inventory plugin. Inventory plugins are off by default and must be enabled explicitly.
- **[connection] pipelining** = **True** — a major Linux performance win: it reduces the number of SSH operations per task. **Important:** in `ansible-core 2.16+` this setting is read from the `[connection]` (or `[defaults]`) section — *not* `[ssh_connection]`, where older guides place it. Pipelining requires `requiretty` to be **off** in the managed hosts' sudoers (the default on modern cloud images).
- **[ssh_connection] ssh_args / control_path** — enable SSH connection multiplexing (ControlMaster/ControlPersist) so repeated connections to the same host reuse one TCP/SSH session. The `%r@%h:%p` tokens are escaped percent signs (INI requires `%` to mean a literal `%`).
- **[privilege_escalation]** — escalate to root via `sudo` by default on Linux tasks. Windows tasks ignore this; WinRM connects as an administrator.

2.4 requirements.yml — the collections

This file lists the Galaxy collections the repository depends on, with version ceilings below the next major release so an unattended reconverge cannot pull a breaking change:

```
collections:
- name: amazon.aws                # aws_ec2 inventory + aws_secret lookup
  version: ">=6.0.0,<11.0.0"
- name: ansible.windows           # win_* core modules
  version: ">=2.0.0,<4.0.0"
- name: microsoft.ad              # AD DS forest/domain promotion
  version: ">=1.0.0,<2.0.0"
- name: community.windows         # extended Windows modules
  version: ">=2.0.0,<4.0.0"
- name: chocolatey.chocolatey
  version: ">=1.4.0,<2.0.0"
- name: ansible.posix
```

```

version: ">=1.5.0,<3.0.0"
- name: community.general
  version: ">=7.0.0,<12.0.0"
- name: community.mysql          # MySQL modules
  version: ">=3.0.0,<4.0.0"

```

What each provides in this repository:

Collection	Used for
amazon.aws	EC2 dynamic inventory; aws_secret lookup
ansible.windows	win_feature, win_service, win_package, win_copy, win_get_url, win_share, win_acl, win_reboot, win_dns_client
microsoft.ad	microsoft.ad.domain (forest), microsoft.ad.membership (join)
community.windows	win_timezone (baseline role)
chocolatey.chocolatey	win_chocolatey, win_chocolatey_source
ansible.posix	POSIX helpers (available for future tasks)
community.general	timezone, ansible_galaxy_install
community.mysql	mysql_user (optional root-password step)

The version ranges are deliberately conservative. To pin exactly for reproducible builds, run `ansible-galaxy collection list` after installing and replace the ranges with the resolved versions.

2.5 Dynamic inventory — `inventory/aws_ec2.yml`

This file configures the `amazon.aws.aws_ec2` inventory plugin. Instead of a static host list, the plugin queries EC2 every run and builds the inventory live.

```

plugin: amazon.aws.aws_ec2
regions:
  - "{{ lookup('ansible.builtin.env', 'AWS_REGION') | default('eu-west-3', true) }}"
filters:
  instance-state-name: running
  tag:ManagedBy: terraform
hostnames:
  - private-ip-address
compose:
  ansible_host: private_ip_address
keyed_groups:
  - key: tags.OS
    prefix: os
    separator: "_"
  - key: tags.Role
    prefix: role
  - key: tags.Environment
    prefix: env

```

Reading it top to bottom:

- **plugin: `amazon.aws.aws_ec2`** — selects the EC2 inventory plugin (enabled in `ansible.cfg`).

- **regions** — which region(s) to query. The value is templated: it reads the `AWS_REGION` environment variable and falls back to `eu-west-3` if unset. The true second argument to `default()` makes the fallback apply to an *empty* string as well as an undefined one. This makes the region a single, overridable knob shared with the playbooks and `reconverge.sh`.
- **filters** — restrict discovery to instances that are **running** and carry the tag `ManagedBy: terraform`. Untagged or stopped instances are ignored, so the inventory only ever contains intentionally-managed hosts.
- **hostnames: private-ip-address** — name each host by its private IP. The control node lives in the VPC, so private addressing is the correct, routable identity.
- **compose: ansible_host: private_ip_address** — set the connection address to the private IP explicitly.
- **keyed_groups** — the heart of tag-driven targeting. For each instance the plugin creates groups from tag values:
 - `tags.OS` with prefix `os` and separator `_` → groups like `os_linux`, `os_windows`.
 - `tags.Role` with prefix `role` → `role_web`, `role_dc`, `role_fileserver`, `role_db`, ...
 - `tags.Environment` with prefix `env` → `env_prod`, `env_dev`, ...

The default separator is `_`, which is why `role/env` groups also use an underscore even though only the `OS` key sets it explicitly.

Verify discovery on the control node with:

```
ansible-inventory -i inventory/aws_ec2.yml --graph
```

You should see `os_linux` and `os_windows` populated. The matching `group_vars/os_linux.yml` and `group_vars/os_windows.yml` then attach the right connection settings automatically, purely because the group names line up.

2.6 Variables — the `group_vars/` directory

Ansible automatically loads `group_vars/<groupname>.yml` for any group a host belongs to. Because the inventory creates `os_linux` / `os_windows` groups, the matching files apply their connection settings to exactly the right hosts.

`group_vars/all.yml` — applies to every host

```
ansible_ssh_private_key_file: /etc/ansible/keys/ansible_ed25519
aws_region: "{{ lookup('ansible.builtin.env', 'AWS_REGION') | default('eu-west-3', true)
↪ }}"

# Name of the ONE consolidated credentials secret (Terraform module.secrets);
# the dynamic name is injected by the control node's cloud-init as ANSIBLE_SECRET_NAME.
ansible_secret_name: "{{ lookup('ansible.builtin.env', 'ANSIBLE_SECRET_NAME') |
↪ default('', true) }}"

# Resolve it ONCE into a JSON object: ssh_private_key / winrm_username / winrm_password
ansible_credentials: "{{ lookup('amazon.aws.aws_secret', ansible_secret_name,
↪ region=aws_region) | from_json }}"

repo_base_url: http://repo.internal.example.com
```

- **ansible_ssh_private_key_file** — the private key used for SSH, written to `/etc/ansible/keys/ansible_` by `bootstrap.yml` from the consolidated secret (see Chapter 2.9).
- **aws_region** — the region used by `aws_secret` lookups and elsewhere. It reads the same `AWS_REGION` env var as the inventory, so region is configured in exactly one place.
- **ansible_secret_name** — the name of the single consolidated credentials secret. Terraform

creates the secret with a dynamic name; because Terraform and Ansible run on different machines, the name is handed to the control node by its cloud-init as the `ANSIBLE_SECRET_NAME` environment variable.

- **ansible_credentials** — resolves that secret **once** into a JSON object with `ssh_private_key`, `winrm_username`, and `winrm_password`. Evaluated lazily, so Linux-only runs never call it. `os_windows.yml` and `bootstrap.yml` read fields off this single variable.
- **repo_base_url** — the internal package-repository server. Playbooks and the `site.yml` repo tasks reference this instead of public mirrors, keeping the estate on the internal supply chain.

group_vars/os_linux.yml — SSH connection + login

```
ansible_connection: ssh
ansible_become: true
ansible_become_method: sudo
linux_login_user: ubuntu
ansible_user: "{{ linux_login_user }}"
# ansible_ssh_common_args: >-
#   -o ProxyCommand="ssh -W %h:%p -q ubuntu@{{ bastion_host }}"
```

- **ansible_connection: ssh** plus **ansible_become/ansible_become_method** — connect over SSH and escalate with sudo.
- **linux_login_user / ansible_user** — the SSH login user. It *must* be known before Ansible connects (it cannot be auto-detected from facts, which are gathered only after connecting), so it defaults to `ubuntu` and is indirected through `linux_login_user` for easy override. For mixed AMIs set `linux_login_user` per host or group (e.g. `ec2-user` for Amazon Linux/RHEL, `rocky` for Rocky Linux) in a `host_vars` file, an `env_*/role_*` group_vars file, or via a distro tag and keyed group.
- The commented **ansible_ssh_common_args** shows the optional bastion `ProxyCommand` for cases where the control node cannot route directly to a host. In this lab's topology (control node inside the VPC) it is not needed.

group_vars/os_windows.yml — WinRM connection (consolidated secret)

```
ansible_connection: winrm
ansible_port: 5986
ansible_winrm_transport: ntlm
ansible_winrm_server_cert_validation: validate
ansible_winrm_message_encryption: auto

# Credentials come from the consolidated secret resolved in group_vars/all.yml.
ansible_user: "{{ ansible_credentials.winrm_username }}"
ansible_password: "{{ ansible_credentials.winrm_password }}"
```

- **ansible_connection: winrm**, **ansible_port: 5986** — connect over WinRM on the HTTPS listener.
- **ansible_winrm_transport: ntlm** — NTLM authentication (use `kerberos` if the hosts are domain-joined).
- **ansible_winrm_server_cert_validation: validate** — validate the listener's TLS certificate in production; only relax to `ignore` against a self-signed listener in a lab.
- **ansible_winrm_message_encryption: auto** — encrypt the WinRM payload.
- **ansible_user / ansible_password** — read directly from `ansible_credentials` (the consolidated secret resolved in `group_vars/all.yml`) as `.winrm_username / .winrm_password`. Nothing is stored in Git, and `ansible_password` is automatically scrubbed from Ansible's output.

host_vars/ — per-host overrides

The `host_vars/` directory holds optional per-host variable files named after an inventory host-name (here, a private IP). It currently contains only `.gitkeep` (a zero-content placeholder so the otherwise-empty directory is tracked by Git). Per-host overrides are rare; `group_vars` covers almost everything.

2.7 Variable precedence (why the right value wins)

When the same variable is set in multiple places, Ansible resolves it by a fixed precedence order. The places this repo uses, from lowest to highest priority:

1. **Role defaults** (`roles/baseline/defaults/main.yml`) — the weakest; meant to be overridden.
2. **Inventory group_vars** (`group_vars/all.yml`, then more specific groups).
3. **Play vars** (a `vars:` block inside a playbook).
4. **Extra vars** (`-e` on the command line) — the strongest; always wins.

This is why a playbook's `vars:` default (say `share_name: data`) is overridden by `-e share_name=software` at run time, and why role defaults are safe to set without fear of clobbering operator intent.

2.8 The baseline role

A **role** packages reusable configuration. Ansible recognizes a role by its conventional subdirectories: `tasks/` (what to do), `defaults/` (overridable variables), `meta/` (metadata and dependencies), plus optional `files/`, `templates/`, `handlers/`, and `vars/`. This repo's single role, `baseline`, applies a minimal cross-platform baseline (timezone, a few packages, time-sync) and is included by `site.yml` for both Linux and Windows.

roles/baseline/tasks/main.yml — the dispatch entry point

```
- name: Include Linux baseline tasks
  ansible.builtin.include_tasks: linux.yml
  when: ansible_os_family != "Windows"

- name: Include Windows baseline tasks
  ansible.builtin.include_tasks: windows.yml
  when: ansible_os_family == "Windows"
```

`tasks/main.yml` is the file Ansible runs first when a role is invoked. Here it is a pure dispatcher: it includes `linux.yml` on non-Windows hosts and `windows.yml` on Windows hosts, keyed on the `ansible_os_family` fact. This keeps the role usable from either OS-scoped play without duplicating logic.

roles/baseline/tasks/linux.yml

```
- name: Ensure a consistent timezone
  community.general.timezone:
    name: "{{ baseline_timezone }}"

- name: Ensure baseline packages are present
  ansible.builtin.package:
    name: "{{ baseline_linux_packages }}"
    state: present
```

```
- name: Ensure chrony (time sync) is enabled and running
  ansible.builtin.service:
    name: "{{ baseline_chrony_service }}"
    state: started
    enabled: true
    when: baseline_manage_timesync | bool
```

Sets the timezone, installs the baseline package list using the generic package module (which resolves to apt or dnf per host), and ensures the time-sync service runs. `ansible.builtin.package` is used deliberately so the same task works on both Debian and RedHat families.

roles/baseline/tasks/windows.yml

```
- name: Set the timezone
  community.windows.win_timezone:
    timezone: "{{ baseline_windows_timezone }}"

- name: Ensure baseline Chocolatey packages are present
  chocolatey.chocolatey.win_chocolatey:
    name: "{{ baseline_windows_packages }}"
    state: present
    when: baseline_windows_packages | length > 0
```

Sets the Windows timezone and installs baseline Chocolatey packages. The header comment captures an important hardened-host principle: prefer signed installers / signed Chocolatey packages over large custom PowerShell, because under WDAC/AppLocker enforcement (especially Constrained Language Mode) Ansible's staged PowerShell can fail.

roles/baseline/defaults/main.yml

```
baseline_timezone: Asia/Kolkata           # Indian Standard Time (IST)
baseline_windows_timezone: India Standard Time
baseline_manage_timesync: true
baseline_chrony_service: chrony           # 'chronyd' on RHEL/Rocky
baseline_linux_packages:
  - htop
  - curl
  - vim
baseline_windows_packages:
  - 7zip
```

Default values for everything the role references. Because they live in `defaults/`, any `group_vars`, `host_vars`, or `-e` override takes precedence. Note the `baseline_chrony_service` default is `chrony` (Debian's service name); on RHEL/Rocky it should be overridden to `chronyd`.

roles/baseline/meta/main.yml and README.md

`meta/main.yml` carries Galaxy metadata — role name, author (zms-lab), license, minimum Ansible version (2.16), supported platforms (Ubuntu, EL, Windows), and an empty dependencies list (the role depends on no other roles). `README.md` documents the role's purpose, variables, and the hardened-Windows note for anyone reusing it.

2.9 The control-node lifecycle: bootstrap.yml and reconverge.sh

These two files keep the *control node itself* current. They are the bridge between cloud-init and the repository.

scripts/reconverge.sh

```
#!/usr/bin/env bash
set -euo pipefail

REPO_DIR="${CONTROL_REPO_DIR:-/opt/control-repo}"

cd "${REPO_DIR}"
git pull --ff-only
ansible-galaxy collection install -r requirements.yml

# AWS_REGION and ANSIBLE_SECRET_NAME are injected into the environment by the
# control node's cloud-init. bootstrap.yml resolves the consolidated secret and
# writes the SSH key from it.
ansible-playbook bootstrap.yml
```

This shell script is the entry point a cloud-init hook or systemd timer invokes on the control node. With `set -euo pipefail` it fails fast on any error. It pulls the latest control repo (fast-forward only, so a diverged history aborts rather than merging), refreshes collections, and runs `bootstrap.yml` — which resolves the consolidated secret and writes the SSH key itself. `AWS_REGION` and `ANSIBLE_SECRET_NAME` come from the environment (injected by the control node's cloud-init); `REPO_DIR` defaults to `/opt/control-repo`.

bootstrap.yml

```
- name: Bootstrap the Ansible control node
  hosts: localhost
  connection: local
  gather_facts: true
  become: true
  vars:
    ansible_key_dir: /etc/ansible/keys
  tasks:
    - name: Assert the consolidated secret name was provided
      ansible.builtin.assert:
        that:
          - ansible_secret_name | length > 0
        fail_msg: >-
          ANSIBLE_SECRET_NAME is not set; cloud-init must inject it.

    - name: Ensure the Ansible key directory exists
      ansible.builtin.file:
        path: "{{ ansible_key_dir }}"
        state: directory
        owner: root
        group: root
        mode: "0700"

    - name: Install required collections
      community.general.ansible_galaxy_install:
```

```

    type: collection
    requirements_file: "{{ playbook_dir }}/requirements.yml"
    become: false

- name: Write the Ansible SSH private key from the consolidated secret
  ansible.builtin.copy:
    content: "{{ ansible_credentials.ssh_private_key }}"
    dest: "{{ ansible_key_dir }}/ansible_ed25519"
    owner: root
    group: root
    mode: "0600"
    no_log: true

- name: Confirm the dynamic inventory resolves
  ansible.builtin.command: ansible-inventory -i inventory/aws_ec2.yml --graph
  args:
    chdir: "{{ playbook_dir }}"
  changed_when: false
  become: false

```

`bootstrap.yml` is the control node configuring **itself** — note `hosts: localhost` and `connection: local`. It ensures the key directory exists with correct ownership and mode, installs the collections from `requirements.yml`, writes the SSH key from `ansible_credentials.ssh_private_key` (the `aws_secret` lookup runs on the control node; `no_log: true` keeps the key out of logs), and finally runs `ansible-inventory --graph` to confirm discovery works. That last task is marked `changed_when: false` because it only *reads* state and must never report a change. The control node's systemd timer re-applies this playbook on a schedule, making the control node self-healing.

2.10 The estate playbook: `site.yml`

`site.yml` is the estate-wide converge — the playbook you run to bring *all* managed hosts to baseline. It contains two plays, one per OS family.

```

- name: Linux estate
  hosts: os_linux
  gather_facts: true
  serial: "{{ rolling_batch | default('100%') }}"
  max_fail_percentage: "{{ max_fail_pct | default(0) }}"
  roles:
    - role: baseline
      tags: [baseline]
  tasks:
    - name: Configure the internal apt repo (Debian family)
      when: ansible_os_family == "Debian"
      tags: [packages]
      block:
        - name: Install the internal repository signing key
          ansible.builtin.get_url:
            url: "{{ repo_base_url }}/keys/internal-archive-keyring.gpg"
            dest: /usr/share/keyrings/internal-archive-keyring.gpg
            mode: "0644"
            owner: root
            group: root
        - name: Point at the internal apt repo (GPG-verified)
          ansible.builtin.apt_repository:

```

```

    repo: >-
      deb [signed-by=/usr/share/keyrings/internal-archive-keyring.gpg]
        {{ repo_base_url }}/deb stable main
      filename: internal
      state: present
      update_cache: true

- name: Configure the internal yum repo (RHEL family)
  when: ansible_os_family == "RedHat"
  tags: [packages]
  ansible.builtin.yum_repository:
    name: internal
    description: Internal repository
    baseurl: "{{ repo_base_url }}/rpm"
    gpgcheck: true
    gpgkey: "{{ repo_base_url }}/keys/RPM-GPG-KEY-internal"
    enabled: true

- name: Install baseline package
  ansible.builtin.package:
    name: "{{ linux_baseline_package | default('htop') }}"
    state: present
    tags: [packages]

- name: Windows estate
  hosts: os_windows
  gather_facts: true
  serial: "{{ rolling_batch | default('100%') }}"
  max_fail_percentage: "{{ max_fail_pct | default(0) }}"
  roles:
    - role: baseline
      tags: [baseline]
  tasks:
    - name: Configure internal Chocolatey source
      chocolatey.chocolatey.win_chocolatey_source:
        name: internal
        source: "{{ repo_base_url }}/choco"
        state: present
        tags: [packages]

    - name: Install a signed package
      chocolatey.chocolatey.win_chocolatey:
        name: "{{ windows_baseline_package | default('7zip') }}"
        state: present
        tags: [packages]

```

Key points:

- **Two plays, selected by group.** The first targets `os_linux`, the second `os_windows`. Each applies the baseline role, then OS-appropriate package tasks.
- **serial and max_fail_percentage.** Both default to 100% and 0 respectively, i.e. all hosts at once and abort on the first failure. Override at run time for rolling, blast-radius-limited converges: `-e rolling_batch=25% -e max_fail_pct=10`.
- **OS-family branching inside the Linux play.** A when: `ansible_os_family == "Debian"` block configures a GPG-verified internal apt repository (it installs the signing key, then adds the repo with `signed-by=` — deliberately *not* `[trusted=yes]`, which would disable signature checks). A parallel when: `... == "RedHat"` task configures the internal yum repository with `gpgcheck: true`. The final package install uses the generic package module.

- **Windows play.** Configures the internal Chocolatey source, then installs a signed package.
- **Tags.** `baseline` and `packages tags` let you run a slice, e.g. `--tags packages` to only touch repositories and packages.

Part III — The Task Playbooks

The `playbooks/` directory holds twelve standalone, single-purpose playbooks. They are run individually (`ansible-playbook playbooks/<name>.yml`) rather than as part of `site.yml`. A shared convention runs through them:

- **hosts:** `"{{ target | default('<group>') }}"` — each defaults to a sensible group but can be redirected with `-e target=...` or narrowed with `--limit`.
- **OS self-selection** — Linux playbooks end the play on hosts of the wrong family with `ansible.builtin.meta: end_host`, so they are safe to run estate-wide; only matching hosts act.
- **Secrets from Secrets Manager** — any credential is fetched at run time via the `aws_secret` lookup; nothing sensitive is stored in Git.

3.1 windows-adds.yml — build a new AD forest

Purpose. Promote a Windows Server to the first Domain Controller of a brand new forest with root domain **alcor.co.in** (NetBIOS **ALCOR**). **Default target:** `role_dc`.

Tasks, in order:

1. **ansible.windows.win_feature** installs the AD-Domain-Services role plus its management tools.
2. **ansible.windows.win_reboot** reboots if the feature install set `reboot_required`.
3. **microsoft.ad.domain** promotes the host to the first DC of the new forest. It receives `dns_domain_name: alcor.co.in`, `domain_netbios_name: ALCOR`, `install_dns: true`, and `reboot: true` (the module manages the post-promotion reboot). The **safe-mode (DSRM) password** comes from the Secrets Manager secret `ansible-control/adds-dsrm-password`; the module marks it `no_log` automatically.
4. **ansible.windows.win_service** waits for the ADWS (Active Directory Web Services) service to be running, retrying up to 30 times with a 20-second delay — a readiness gate confirming the DC is up after reboot.

Prerequisite: create `ansible-control/adds-dsrm-password` (a strong password, plain string) before running.

3.2 windows-domain-join.yml — join hosts to the domain

Purpose. Join Windows machines to **alcor.co.in**. **Default target:** `os_windows`.

1. **ansible.builtin.meta: end_host** when the host is in `role_dc` — domain controllers *are* the domain and must be skipped.
2. **ansible.windows.win_dns_client** points the host's DNS at the domain controller — but only when `domain_dns_server` is set. Leave it empty if the VPC DHCP option set already serves the DC as DNS.
3. **microsoft.ad.membership** performs the join (`state: domain`, `reboot: true`). The join credentials come from the JSON secret `ansible-control/domain-join-credential` (username / password), an account permitted to add computers to the domain. An optional `ou_path` places the computer object in a specific Organizational Unit.
4. **ansible.builtin.debug** reports the per-host membership result.

Prerequisite: create `ansible-control/domain-join-credential` (JSON with username like

ALCOR\joinadmin and password).

3.3 windows-iis.yml — enable the IIS web server

Purpose. Install IIS on selected hosts. **Default target:** `role_web`.

1. **ansible.windows.win_feature** installs the features listed in `iis_features` (default Web-Server and Web-Mgmt-Console) with management tools.
2. **ansible.windows.win_reboot** reboots if required.
3. **ansible.windows.win_service** ensures the W3SVC (World Wide Web Publishing) service is started and set to start automatically.

Override `iis_features` to add roles, e.g. `-e '{"iis_features":["Web-Server","Web-Asp-Net45","Web-Mgmt-Console"]}'`.

3.4 windows-share.yml — create a read-only SMB share

Purpose. Create an SMB network share with **Everyone read-only** access at both the share and NTFS layers. **Default target:** `role_fileserver`.

1. **ansible.windows.win_file** ensures the share directory exists (default path `C:\Shares\{{ share_name }}`, where `share_name` defaults to `data`).
2. **ansible.windows.win_acl** grants Everyone the NTFS right `ReadAndExecute` (inherited by sub-folders and files). NTFS permissions must allow read for the share to be usable.
3. **ansible.windows.win_share** creates/updates the share with `read: Everyone` — share-level read for everyone, no write.

Override `share_name` / `share_path` to create other shares, e.g. `-e share_name=software -e 'share_path=D:\Shares\software'`.

3.5 windows-python.yml — install Python via Chocolatey

Purpose. Install Python on Windows from the internal Chocolatey source. **Default target:** `os_windows`.

1. **chocolatey.chocolatey.win_chocolatey_source** ensures the internal Chocolatey source exists (pointing at `{{ repo_base_url }}/choco`).
2. **chocolatey.chocolatey.win_chocolatey** installs python3 from that source. The `version` parameter uses `default(omit, true)` so that an empty `python_version` is *omitted* (install latest), while a set value pins the version.
3. **ansible.windows.win_command** runs `python --version` (marked `changed_when: false`), and **ansible.builtin.debug** prints the result.

Pin a version with `-e python_version=3.12.4`.

3.6 windows-zms-enforcer.yml — install the Zscaler ZMS Enforcer

Purpose. An Ansible conversion of the upstream `windows/install.ps1`. It provisions and installs the Zscaler Microsegmentation (ZMS) Enforcer, taking the provisioning **nonce** from Secrets Manager rather than a command-line parameter. **Default target:** `os_windows`.

How each step of the original PowerShell maps to Ansible:

install.ps1 step	Playbook task(s)
Get nonce / write provision_key	aws_secret lookup → win_copy (exact bytes, no_log)
Resolve download endpoint (prod→beta)	win_wait_for loop over the two endpoints; first reachable URL selected
Download MSI with retries	win_get_url with retries/until + validate_certs, then win_stat size check
Install MSI with PROVISIONKEY_FILE	win_package with arguments + expected_return_code: [0, 3010]
Exit-3010 reboot handling	conditional win_reboot (gated by zms_auto_reboot) or a pending-reboot notice

Notable details:

- The nonce is read from `ansible-control/zms-provision-nonce` and written to the `provision_key` file with `win_copy` content: (exact bytes, no BOM or trailing newline). An `assert` first confirms the nonce was retrieved.
- Two endpoints are tried in order — production (`eyez-dist.private.zscaler.com`) then beta (`eyez-dist.zpabeta.net`); the first reachable on port 443 is selected with a Jinja `rejectattr('failed')` expression. If neither is reachable, an `assert` fails the run with a clear message.
- The download is retried up to `zms_download_retries` times; certificate validation (`validate_certs: true`) covers the script's separate TLS-handshake diagnostic — a man-in-the-middle proxy breaking TLS simply fails the download.
- `win_package` accepts both 0 and 3010 (reboot-required) as success.
- A security improvement over the script: the `provision_key` file (which contains the nonce) is **deleted after install** by default (`zms_cleanup_provision_key: true`), so the nonce is not left on disk.

Prerequisite: create `ansible-control/zms-provision-nonce` (plain string, the pipe-delimited nonce from the ZMS Console). Set `-e zms_auto_reboot=true` to let Ansible reboot on exit 3010.

3.7 ubuntu-setup.yml — update + common packages (Ubuntu)

Purpose. A consolidated Ubuntu setup: a full package upgrade *and* the installation of common packages. It replaces three earlier playbooks (update-packages, install-netcat, install-packages).

Default target: `os_linux`. Every host that is not Debian-family ends the play immediately via `meta: end_host`.

Two tagged sections let you run a slice:

- **--tags update** — `ansible.builtin.apt` performs a full dist upgrade (`update_cache: true`, `cache_valid_time: 3600`, `upgrade: dist`), then a second `apt` task runs `autoremove/autoclean`. A stat on `/var/run/reboot-required` decides whether `ansible.builtin.reboot` runs.
- **--tags packages** — `ansible.builtin.apt` installs the `ubuntu_packages` list, which defaults to `netcat-openbsd`, `curl`, and `wget`. Override with `-e '{"ubuntu_packages":["curl","wget","git"]}'`.

The play honors `rolling_batch` and `max_fail_pct` for safe, batched upgrades.

3.8 ubuntu-apache2.yml — Apache2 service (Ubuntu)

Purpose. Install Apache2 and run it as a service on selected Ubuntu hosts. **Default target:** `os_linux` (use `--limit`). Skips non-Debian hosts.

1. **ansible.builtin.apt** installs `apache2` (with cache refresh).
2. **ansible.builtin.service** ensures `apache2` is started and enabled at boot.

3.9 ubuntu-mysql.yml — MySQL service (Ubuntu)

Purpose. Install MySQL and run it as a service on selected Ubuntu hosts. **Default target:** `os_linux` (use `--limit`). Skips non-Debian hosts.

1. **ansible.builtin.apt** installs `mysql-server` and `python3-pymysql` (the Python client the Ansible MySQL modules require).
2. **ansible.builtin.service** ensures the `mysql` service is started and enabled.
3. **community.mysql.mysql_user** optionally sets the root password — but only when `mysql_root_password` is non-empty (default empty, so the step is skipped by default). It connects through the local socket `/run/mysqld/mysqld.sock` and is marked `no_log`. Supply the password ideally from Secrets Manager (a commented `aws_secret` lookup shows how). On a fresh Ubuntu install the root account uses `auth_socket`, so local socket access works without a password.

3.10 amazonlinux-setup.yml — update + common packages (Amazon Linux)

Purpose. The Amazon Linux counterpart of `ubuntu-setup.yml`. **Default target:** `os_linux`; non-RedHat-family hosts end the play immediately.

- **--tags update** — `ansible.builtin.dnf` upgrades all packages (`name: "*" , state: latest`, `update_cache: true`), installs `dnf-utils`, then runs `needs-restarting -r`. That command is changed `when: false` and `failed_when: false`, and its return code (`rc == 1`) drives `ansible.builtin.reboot`.
- **--tags packages** — `ansible.builtin.dnf` installs the `amazonlinux_packages` list, defaulting to `nmap-ncat` (Amazon Linux's `netcat`), `curl`, and `wget`.

Honors `rolling_batch / max_fail_pct`.

3.11 amazonlinux-httpd.yml — Apache (httpd) service (Amazon Linux)

Purpose. Install Apache as **httpd** — the Amazon Linux package/service name; the Debian name `apache2` does not apply here. **Default target:** `os_linux` (use `--limit`). Skips non-RedHat hosts.

1. **ansible.builtin.dnf** installs `httpd`.
2. **ansible.builtin.service** ensures `httpd` is started and enabled.

3.12 amazonlinux-mysql.yml — MySQL service (Amazon Linux)

Purpose. Install MySQL on Amazon Linux. Because Amazon Linux 2023 (EL9-based) does not ship MySQL in its core repositories, the playbook adds the official MySQL Community repository first. **Default target:** `os_linux` (use `--limit`). Skips non-RedHat hosts.

1. **ansible.builtin.dnf** installs the MySQL Community release RPM from `mysql_repo_rpm` (default `mysql84-community-release-el9-1.noarch.rpm`), with `disable_gpg_check: true` for that bootstrap RPM. Point `mysql_repo_rpm` at an internal mirror if preferred.
2. **ansible.builtin.dnf** installs `mysql-community-server` and `python3-PyMySQL`.
3. **ansible.builtin.service** ensures the `mysqld` service is started and enabled.
4. **community.mysql.mysql_user** optionally sets the root password (skipped unless `mysql_root_password` is set), via the socket `/var/lib/mysql/mysql.sock`.

The header notes two real-world caveats: MySQL Community generates a *temporary* root password in `/var/log/mysqld.log` on first start (so non-interactive root setup is best-effort), and MariaDB (`mariadb105-server`, service `mariadb`) is a no-external-repo alternative if a MySQL-compatible drop-in is acceptable.

Part IV — Operating the Estate

4.1 The everyday commands

All commands are run from the repository root so `ansible.cfg` and the dynamic inventory are picked up automatically.

```
# Preview an estate converge (dry run + show diffs), changing nothing
ansible-playbook site.yml --check --diff

# Apply the estate converge
ansible-playbook site.yml

# Limit to a subset of hosts (intersection: prod Linux only)
ansible-playbook site.yml --limit 'os_linux:&env_prod'

# Run only the tasks carrying a tag
ansible-playbook site.yml --tags packages

# Rolling, fail-fast converge (25% of hosts at a time, abort past 10% failures)
ansible-playbook site.yml -e rolling_batch=25% -e max_fail_pct=10

# Run a single task playbook, narrowed to specific hosts
ansible-playbook playbooks/ubuntu-apache2.yml --limit web01,web02

# Ad-hoc connectivity checks
ansible os_linux -m ping
ansible os_windows -m ansible.windows.win_ping
```

A few mechanics worth internalizing:

- **--check is your seat belt.** It reports changed where a real run *would* act, without acting. Combine with `--diff` to see file/line changes. Some modules cannot fully predict changes in check mode, but it catches most drift.
- **--limit patterns** support set arithmetic: `:` is union, `&` is intersection, `!` is exclusion. `os_linux:&env_prod` means “Linux *and* prod.”
- **--tags / --skip-tags** select task subsets. This repo tags baseline and packages in `site.yml`, and update / packages in the `*-setup` playbooks.
- **-e (extra vars)** override anything and are the right way to pass run-time choices such as `rolling_batch`, `target`, `share_name`, or `python_version`.

4.2 Targeting model recap

Targeting flows from cloud tags through the inventory to the playbooks:

- Tag an instance `OS=linux` or `OS=windows` → it lands in `os_linux` / `os_windows`.
- Tag it `Role=web`, `Role=dc`, `Role=fileserver`, `Role=db` → it lands in `role_web`, `role_dc`, `role_fileserver`, `role_db`.
- Tag it `Environment=prod/dev` → it lands in `env_prod` / `env_dev`.

Playbooks default to a group but accept `-e target=<group>` or `--limit <pattern>`. The role-group playbooks (`ADDS`→`role_dc`, `IIS`→`role_web`, `share`→`role_fileserver`) assume you

have tagged those instances accordingly; if you have not, use `--limit` with explicit hostnames.

4.3 Secrets the repository expects

Nothing sensitive lives in Git. Create these AWS Secrets Manager secrets before running the playbooks that need them. The region is the one from `AWS_REGION` (default `eu-west-3`).

The **connection credentials** (SSH key + WinRM) live in a single consolidated secret created by Terraform (`module.secrets`), named `<base>/ansible-credentials-<suffix>`. Because Terraform and Ansible run on different machines, its dynamic name reaches the control node via the `ANSIBLE_SECRET_NAME` environment variable (cloud-init), and `group_vars/all.yml` resolves it once into `ansible_credentials`. The per-playbook secrets remain separate.

Secret	Format	Used by
<code><base>/ansible-credentials-ssh</code> (name via <code>ANSIBLE_SECRET_NAME</code>)	JSON: <code>{ssh_private_key, winrm_username, winrm_password}</code>	SSH key (<code>bootstrap.yml</code>) + WinRM (<code>os_windows.yml</code>)
<code>ansible-control/adds-dsrm-password</code>	plain string	<code>windows-adds.yml</code>
<code>ansible-control/domain-join-credentials</code>	JSON: <code>{username, password}</code>	<code>windows-domain-join.yml</code>
<code>ansible-control/zms-provisioning-password</code>	plain string	<code>windows-zms-enforcer.yml</code>
<code>ansible-control/mysql-root-password</code>	plain string (optional)	<code>ubuntu-mysql.yml</code> , <code>amazonlinux-mysql.yml</code>

The control node's IAM role must be permitted to read these (the connection secret's `GetSecretValue` is scoped to that one ARN). JSON secrets are parsed with the `from_json` filter; string secrets are used as-is.

4.4 Drift cadence

The control node's systemd timer re-applies `bootstrap.yml` to itself, keeping the control node current. To converge the *estate* on a schedule, add a second timer that runs `ansible-playbook site.yml` — ideally `--check` first with alerting on any changed, then enforce. At a few dozen hosts a 30-60 minute cadence is comfortable.

4.5 Linting and CI

Quality gates keep the repository healthy and catch regressions (such as a mis-sectioned `ansible.cfg` setting) before they reach the estate.

- **.ansible-lint** sets profile: `production` and excludes `collections/`, `.github/`, and `.cache/` from linting.
- **.yamllint** extends the default rules, relaxing line length to a 160-column warning and restricting truthy values to `true/false`.
- **.pre-commit-config.yaml** wires `yamllint` and `ansible-lint` as git pre-commit hooks (pre-commit install to activate). They run on every commit.
- **.github/workflows/lint.yml** runs the same checks in CI on push and pull request: it checks out the repo, sets up Python 3.12, installs `ansible-core/ansible-lint/yamllint`, installs the collections, and runs `yamllint .` then `ansible-lint`.

Run them locally with:

```
pip install ansible-lint yamllint pre-commit
pre-commit install
ansible-lint
yamllint .
```

4.6 What .gitignore protects

.gitignore deliberately keeps three classes of file out of version control: the installed collections/ directory (reproduced from requirements.yml); the fact cache and .retry files; and — most importantly — **secret material**: keys/, *.pem, *_ed25519, *_rsa, .vault_pass, and vault_pass.txt. This is the on-disk half of the “no secrets in Git” rule; the runtime half is the aws_secret lookup.

Part V — Hands-On Training Course

This part turns the reference material into a guided course. Each module has an objective, prerequisites, steps, and a verification check. Do them in order on a disposable lab estate. None of these are destructive when run with `--check` first, which every module begins with.

Module 1 — Orientation and a safe first run

Objective. Install collections, confirm discovery, and preview a converge without changing anything.

Steps.

1. From the repo root, install collections: `ansible-galaxy collection install -r requirements.yml`.
2. Set the region if it differs from the default: `export AWS_REGION=eu-west-3`.
3. Confirm discovery: `ansible-inventory -i inventory/aws_ec2.yml --graph`.
4. Preview the estate converge: `ansible-playbook site.yml --check --diff`.

Verification. The graph shows `os_linux` and `os_windows` populated, and the `--check` run completes with a recap and no errors. Note which hosts report changed — those are where a real run would act.

Module 2 — Reading the inventory and groups

Objective. Understand how tags become groups.

Steps.

1. List every group: `ansible-inventory --graph`.
2. List a single group's hosts: `ansible-inventory --graph os_linux`.
3. Dump one host's variables: `ansible-inventory --host <private-ip>`.

Verification. You can explain why a given instance is in `os_linux`, `role_web`, and `env_prod` simultaneously (its OS, Role, and Environment tags), and you can see the `ansible_host` set to its private IP.

Module 3 — Connectivity (ping) to both OS families

Objective. Confirm Ansible can reach Linux and Windows.

Steps.

1. `ansible os_linux -m ping` (returns pong over SSH).
2. `ansible os_windows -m ansible.windows.win_ping` (returns pong over WinRM).

Verification. Both return pong. If Linux fails, check the SSH key was fetched to `/etc/ansible/keys/`; if Windows fails, check the WinRM listener and that the consolidated secret (`ANSIBLE_SECRET_NAME`) resolves.

Module 4 — A tagged, scoped converge

Objective. Use tags and limits to act narrowly.

Steps.

1. Preview only package tasks on prod Linux: `ansible-playbook site.yml --tags packages --limit 'os_linux:&env_prod' --check`.
2. Apply it (drop `--check`) once the preview looks right.

Verification. Only the repository and package tasks ran, and only on prod Linux hosts. Windows hosts and non-prod hosts were untouched.

Module 5 — Install a service on selected hosts

Objective. Stand up Apache on two web hosts and verify idempotency.

Steps.

1. `ansible-playbook playbooks/ubuntu-apache2.yml --limit web01,web02 --check`.
2. Apply it.
3. Run it a **second** time.

Verification. The second run reports ok (not changed) for the install and service tasks — proof of idempotency. `systemctl is-active apache2` on the hosts returns active.

Module 6 — A consolidated update with tags

Objective. Use `ubuntu-setup.yml` to update and to install packages separately.

Steps.

1. Update only, batched: `ansible-playbook playbooks/ubuntu-setup.yml --tags update -e rolling_batch=25% --check`, then apply.
2. Packages only, with an override: `ansible-playbook playbooks/ubuntu-setup.yml --tags packages -e '{"ubuntu_packages":["curl","wget","git"]}'`.

Verification. With `--tags update`, no package-install task ran; with `--tags packages`, no upgrade task ran. `git` is now present on the targeted hosts.

Module 7 — A secret-backed Windows workflow

Objective. Run a playbook that depends on Secrets Manager.

Steps.

1. Create `ansible-control/zms-provision-nonce` with a test nonce string.
2. Preview: `ansible-playbook playbooks/windows-zms-enforcer.yml --limit <win-host> --check`.

Verification. The play's opening assert passes (the nonce resolved). You can articulate why the nonce never appears in logs (`no_log: true` on the `provision_key` write) and why the file is removed afterward.

Module 8 — Capstone

Objective. Tie it together: tag a new Ubuntu instance, discover it, baseline it, and install a service — all without editing the repo.

Steps.

1. Launch/tag an instance `ManagedBy=terraform, OS=linux, Role=web, Environment=dev`.
2. Confirm it appears: `ansible-inventory --graph os_linux`.
3. Baseline it: `ansible-playbook site.yml --limit <new-ip> --check`, then apply.

4. Install Apache: `ansible-playbook playbooks/ubuntu-apache2.yml --limit <new-ip>`.

Verification. The instance was configured purely by virtue of its tags and a `--limit`; you never edited an inventory file.

5.1 Self-check quiz

Answers follow in 5.2.

1. In which `ansible.cfg` section must pipelining appear for ansible-core 2.16+, and why does this matter?
2. What two conditions must an EC2 instance meet to appear in the inventory?
3. Which fact do the playbooks branch on to choose apt vs dnf vs Windows behaviour?
4. Why is `linux_login_user` a variable rather than auto-detected from facts?
5. What does `serial: 25%` do, and which variable sets it in `site.yml`?
6. Where does the Windows WinRM password come from, and in what format is the secret?
7. Why is the `needs-restarting` command marked `changed_when: false`?
8. What is the difference between `state: latest` and `state: present`, and which does the update play use?
9. Why does `windows-domain-join.yml` skip hosts in `role_dc`?
10. What does `default('eu-west-3', true)` accomplish in the region lookup that `default('eu-west-3')` would not?

5.2 Quiz answers

1. The **[connection]** section (or **[defaults]**). In ansible-core 2.16+ the setting is no longer read from **[ssh_connection]**; placing it there silently leaves pipelining off, losing the biggest Linux push speedup.
2. It must be **running** and carry the tag **ManagedBy: terraform** (the inventory filters).
3. **ansible_os_family** — “Debian”, “RedHat”, or “Windows”.
4. Because the SSH login user must be known **before** Ansible connects, and facts are only gathered **after** connecting. So it cannot be auto-detected; it defaults to `ubuntu` and is overridable per host/group.
5. It processes hosts in batches of 25% of the play’s hosts (rolling updates, limiting blast radius). In `site.yml` it is set by `rolling_batch (-e rolling_batch=25%)`.
6. From the single **consolidated** Secrets Manager secret whose name is injected as **ANSIBLE_SECRET_NAME**; it is a **JSON** document (`{ssh_private_key, winrm_username, winrm_password}`) resolved once into `ansible_credentials` in `group_vars/all.yml`, from which `.winrm_username/.winrm_password` are read.
7. Because it only **reads** state (it reports whether a reboot is needed) and must never be counted as a change; otherwise every run would falsely report changed.
8. `state: present` ensures the package is installed (any version); `state: latest` ensures it is the newest available (upgrading if needed). The update play uses **state: latest** with `name: "*" to upgrade everything`.
9. Because a domain controller **is** the domain — it cannot “join” itself. `meta: end_host` skips `role_dc` hosts so the play is safe to run across `os_windows`.
10. The true second argument makes the default apply when the variable is an **empty string**, not only when it is undefined — so `AWS_REGION=""` still falls back to `eu-west-3`.

Appendix A — Command Cheat Sheet

```
# --- Setup ---
ansible-galaxy collection install -r requirements.yml    # install collections
export AWS_REGION=eu-west-3                            # set region (optional)
ansible-inventory -i inventory/aws_ec2.yml --graph      # confirm discovery

# --- Connectivity ---
ansible os_linux -m ping
ansible os_windows -m ansible.windows.win_ping

# --- Estate converge ---
ansible-playbook site.yml --check --diff                # dry run
ansible-playbook site.yml                              # enforce
ansible-playbook site.yml --tags packages               # only tagged tasks
ansible-playbook site.yml --limit 'os_linux:&env_prod'  # intersection
ansible-playbook site.yml -e rolling_batch=25% -e max_fail_pct=10

# --- Task playbooks (examples) ---
ansible-playbook playbooks/windows-adds.yml --limit win-dc01
ansible-playbook playbooks/windows-iis.yml -e target=role_web --check
ansible-playbook playbooks/windows-share.yml -e share_name=software
ansible-playbook playbooks/windows-python.yml -e python_version=3.12.4
ansible-playbook playbooks/windows-domain-join.yml -e domain_dns_server=10.0.0.10
ansible-playbook playbooks/windows-zms-enforcer.yml -e zms_auto_reboot=true
ansible-playbook playbooks/ubuntu-setup.yml --tags update
ansible-playbook playbooks/ubuntu-apache2.yml --limit web01,web02
ansible-playbook playbooks/ubuntu-mysql.yml -e target=role_db
ansible-playbook playbooks/amazonlinux-setup.yml --tags packages
ansible-playbook playbooks/amazonlinux-httpd.yml --limit web01
ansible-playbook playbooks/amazonlinux-mysql.yml -e target=role_db

# --- Quality gates ---
ansible-lint
yamllint .
pre-commit install
```

Appendix B — --limit pattern arithmetic

Pattern	Meaning
os_linux	all hosts in the os_linux group
os_linux:env_prod	union — in either group
os_linux:&env_prod	intersection — in both groups
os_linux:!env_prod	exclusion — Linux but not prod
web01,web02	two explicit hosts
'role_web:&env_prod'	prod web hosts only

Quote patterns containing &, !, or : so the shell does not interpret them.

Appendix C — Glossary

Become — privilege escalation (here, sudo to root on Linux). **Collection** — a versioned bundle of modules/plugins/roles from Galaxy. **Converge** — a run that brings hosts to the desired state. **Drift** — divergence of a host from its declared state over time. **Fact** — auto-gathered host data (ansible_os_family, etc.). **FQCN** — fully-qualified collection name, e.g. ansible.builtin apt. **Handler** — a task that runs only when notified by a changed task. **Idempotent** — repeatable without additional effect once converged. **Inventory** — the (here dynamic) list of hosts and groups. **Keyed group** — an inventory group generated from a tag value. **Lookup** — a control-node plugin that fetches data at run time. **Module** — the unit of work executed on a host. **Play** — a hosts→tasks mapping. **Playbook** — a file of one or more plays. **Role** — a structured, reusable bundle of tasks and variables. **Tag** — a label for selecting task subsets. **Template (Jinja2)** — `{{ }}` expression evaluation.

Appendix D — Troubleshooting

Failed to load inventory plugin aws_ec2. The `amazon.aws` collection or `boto3/botocore` is missing on the control node, or the plugin is not enabled. Confirm `enable_plugins = aws_ec2` in `ansible.cfg`, run `ansible-galaxy collection install -r requirements.yml`, and ensure `boto3` is installed.

Inventory is empty. Check that instances are **running**, carry `ManagedBy=terraform`, and live in the region named by `AWS_REGION`. The control node's IAM role must allow `ec2:Describe*`.

Linux ping fails. The SSH key may not have been fetched. Confirm `/etc/ansible/keys/ansible_ed25519` exists (0600) and that the consolidated secret (`ANSIBLE_SECRET_NAME`) resolves and contains `ssh_private_key`. Check `linux_login_user` matches the AMI (ubuntu vs ec2-user vs rocky).

Windows ping fails. Confirm the WinRM HTTPS listener on 5986, that the consolidated secret (`ANSIBLE_SECRET_NAME`) resolves with valid `winrm_username/winrm_password`, and that certificate validation matches your listener (validate vs ignore).

A secret won't resolve. Verify the secret id exactly, the region, and that the control node's IAM role can read it. JSON secrets must be valid JSON for `from_json` to parse.

Pipelining seems to have no effect. Confirm it is under `[connection]` (not `[ssh_connection]`) and that managed hosts have `requiretty` off in `sudoers`.

A run reports changed every time on an unchanged host. Look for a raw `command/shell` task missing `changed_when: false`, or a non-idempotent step.

Appendix E — File Index

Path	Purpose
ansible.cfg	Engine configuration
requirements.yml	Galaxy collections + version ranges
inventory/aws_ec2.yml	EC2 dynamic inventory config
group_vars/all.yml	Variables for every host
group_vars/os_linux.yml	SSH connection + login user
group_vars/os_windows.yml	WinRM connection + secret lookup
host_vars/.gitkeep	Placeholder for per-host overrides
roles/baseline/tasks/main.yml	OS dispatch entry point
roles/baseline/tasks/linux.yml	Linux baseline tasks
roles/baseline/tasks/windows.yml	Windows baseline tasks
roles/baseline/defaults/main.yml	Role default variables
roles/baseline/meta/main.yml	Role Galaxy metadata
roles/baseline/README.md	Role documentation
scripts/reconverge.sh	Cloud-init / timer entry point
bootstrap.yml	Control node configuring itself
site.yml	Estate-wide converge (2 plays)
playbooks/windows-adds.yml	New AD forest (alcor.co.in)
playbooks/windows-domain-join.yml	Join hosts to the domain
playbooks/windows-iis.yml	Enable IIS
playbooks/windows-share.yml	Read-only SMB share
playbooks/windows-python.yml	Install Python via Chocolatey
playbooks/windows-zms-enforcer.yml	Install Zscaler ZMS Enforcer
playbooks/ubuntu-setup.yml	Update + common packages (Ubuntu)
playbooks/ubuntu-apache2.yml	Apache2 service (Ubuntu)
playbooks/ubuntu-mysql.yml	MySQL service (Ubuntu)
playbooks/amazonlinux-setup.yml	Update + common packages (AL)
playbooks/amazonlinux-httpd.yml	httpd service (Amazon Linux)
playbooks/amazonlinux-mysql.yml	MySQL service (Amazon Linux)
.ansible-lint / .yamllint	Lint configuration
.pre-commit-config.yml	Local git hooks
.github/workflows/lint.yml	CI lint workflow
.gitignore	Never-commit list

End of guide. Keep this document beside the repository — every command and value here

matches the code as written.